Rapport

Zoidberg 2.0

I.A capable de détecter la pneumonie à partir d'images radiographiques

Sommaire

Introduction 3
Organisation 3
Qu'est-ce que le Machine-Learning 4
Le Machine-Learning 4
De l'apprentissage à la prédiction 5
Risque & Fonction de perte 6
Performances 6
Validation 7
Gestion des données 7
Underfitting & Overfitting 8
Qu'est-ce que le Deep-Learning 9
Un neurone 9
Des neurones 11
Entraîner un réseau de neurones 12
Les réseaux de convolution 13
Couche de convolution 13
Couche de pooling 14
Annexe 16

Introduction

La détection des pathologies pulmonaires représente un défi majeur pour les professionnels de santé. Les médecins ont le plus souvent recours à la radiographie thoracique pour établir un diagnostic. Cependant, son interprétation peut être difficile et nécessite souvent une expertise considérable. Dans ce contexte, notre objectif est d'améliorer l'interprétation des radiographies en proposant une solution basée sur le machine learning.

Nous devons ainsi développer un modèle de classification d'images de poumons. Nous avons à notre disposition une base de données d'environs ~6000 images classées en trois catégories : personne saine, pneumonie virale et pneumonie bactérienne. Pour atteindre notre objectif de classification, nous avons opté pour l'utilisation de réseaux de convolutions, une approche qui a montré de très bons résultats dans la classification d'images en extrayant les caractéristiques significatives des images à l'aide de filtres. Nous travaillerons avec des bibliothèques populaires telles que TensorFlow[1] et Keras[2] pour développer notre modèle de classification.

Organisation

Nous avons organisé notre projet autour de trois parties majeures, qui sont essentielles pour tout projet de machine learning : l'analyse des données, la comparaison des différents modèles en vue de sélectionner celui qui obtient les meilleures performances, et enfin l'optimisation du modèle sélectionné.

Qu’est-ce que le Machine-Learning

Le Machine-Learning

Dans plusieurs domaines dans lesquelles l’informatique est utilisée, il peut être utile que l’on ne décrive pas explicitement les règles d’un algorithme mais plutôt que l’ordinateur les apprennent par lui-même. C’est exactement ce que l’on cherche à faire quand on parle de Machine Learning (ou apprentissage automatisé en français).

La définition de T. Mitchell fournie une explication plus pratique de ce domaine :

> “A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.” Tom Mitchell, 1997

Illustrons ça tout de suite avec un des exemples le plus populaire : La classification des emails en spam ou non-spam.

Pour déterminer si un email est un spam, on pourrait tenter de définir des règles explicites sur le titre du mail, une blacklist de mots, etc. Cependant, l’algorithme serait excessivement grand si l’on veut qu’il soit efficace, et sa maintenance serait très complexe au fur et à mesure que de nouveaux spams soient créés. C’est là que l’on se rend compte qu’il nous faut passer par le machine learning. En reprenant la définition de T. Mitchell, nous aurions :

- La tâche T : classifier un email en spam ou non-spam
- L’expérience E : examiner une banque de données d’un ensemble d’emails étiquetés comme spam ou non-spam
- La performance P : la fraction d’emails correctement classés comme spam par l’algorithme.

Il existe de nombreux modèles de machine learning que l’on peut classer en sous-domaines présentés dans la Figure

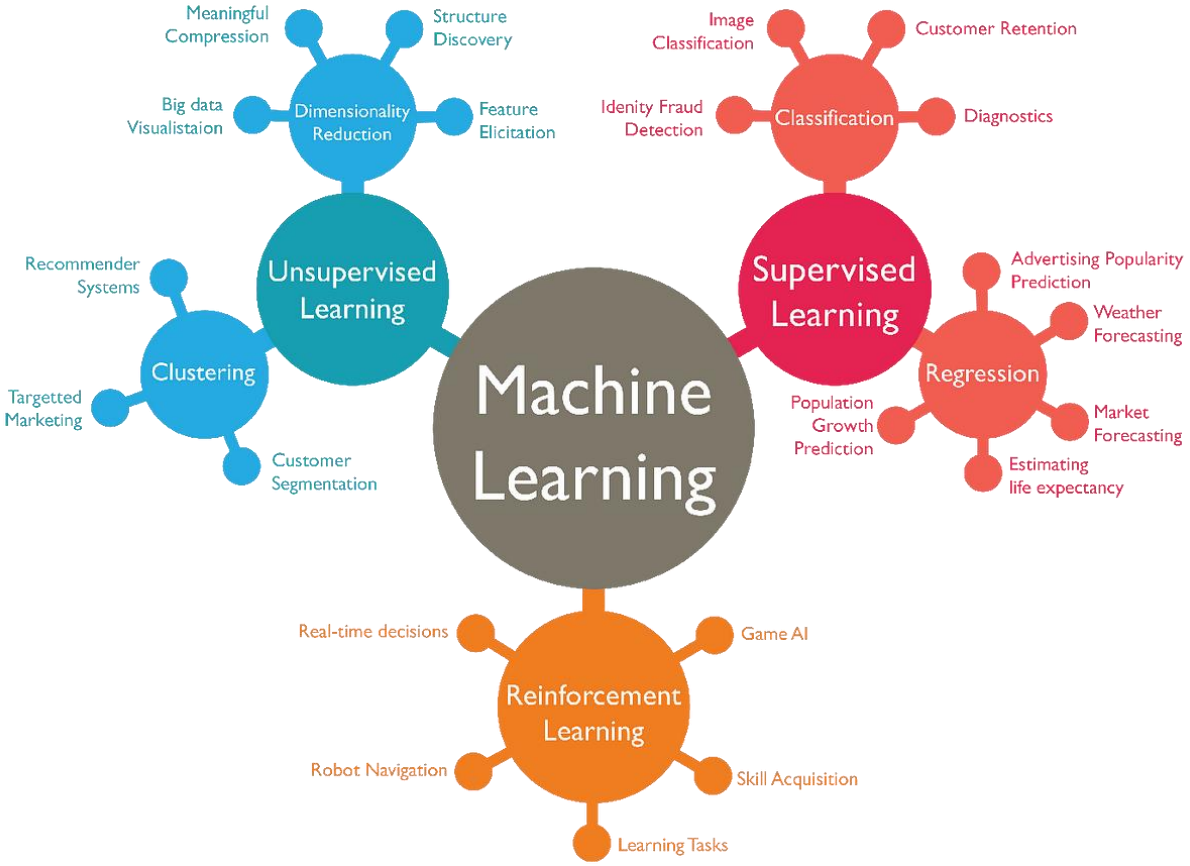


Figure 1. Cartographie non exhaustive des modèles de machine learning

Comme énoncé précédemment, dans ce projet, nous devons réaliser une classification en 3 classes. Nous allons donc nous restreindre à l'apprentissage supervisé, et en particulier à la classification[3].

De l'apprentissage à la prédiction

La problématique de l'apprentissage supervisé :

A partir d'observations données $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}, 1 \leq i \leq m$ de entées[4]/sorties (appelées données d'entraînements), notre but est de prédire un nouveau label $y \in \mathcal{Y}$ à partir d'une nouvelle donnée $x \in \mathcal{X}$(ces nouvelles données étant appelées données de test).

La prédiction n'est autre qu'une fonction dépendant de paramètres θ appelé fonction d'hypothèse :

$$h_\theta : \mathcal{X} \to \mathcal{Y}$$

Pour ZoidBerg 2.0, X est l'ensemble des images et Y ={ normal, virus, bacteria }. La forme de h_θ dépend du modèle choisie[5] : cela peut-être un simple polynôme dans le cas de la régression linéaire ou une fonction plus complexe comme… un réseau de

neurones (détaillé à la section 3). En revanche, θ est justement ce que l'on cherche à trouver via l'apprentissage.

En résumé, on cherche à trouver un algorithme A tel que A prend en entrée la distribution $\mathcal{D}(x_i)$ et nous renvoie θ tout en cherchant à ce que $h_\theta(x_i)$ soit le plus proche de y_i.

Risque & Fonction de perte

Le cœur de l'apprentissage supervisé réside dans le fait de mesurer à quel point la valeur prédite d'une donnée d'entraînement est éloignée de sa valeur réelle. Pour quantifier cet écart, on définit la fonction de perte (loss function en anglais) :

$$L: \mathcal{Y} \times \mathcal{Y} \to R^+, \quad (\overline{y}, y) \mapsto L(\overline{y}, y)$$

Avec $\overline{y_i} = h_\theta(x_i), L(\overline{y_i}, y_i)$ tend vers zéro si la prédiction est proche de la valeur réelle et tend vers l'infini dans le cas contraire.

On cherche un algorithme qui va minimiser cet écart entre "prédit" et "réelle" sur l'ensemble des données d'entraînement. C'est pourquoi on va regarder la moyenne de la fonction de perte sur l'ensemble des données. Cette moyenne étant appelée risque :

$$R_A[\mathcal{D}(x_i)] \coloneqq E[L(h_\theta(x_i), y_i)] = \frac{1}{m}\sum_{i=1}^{m} L(h_\theta(x_i), y_i)$$

Ainsi, l'algorithme A cherche $\tilde{0}$ tel que :

$$\tilde{0} = \arg\min_{\theta} R_A\left[\mathcal{D}(x_i)\right]$$

La définition de ce risque[6] permet de transposer notre problématique d'apprentissage en un problème d'optimisation. En effet, tout en gardant une très grande capacité de généralisation, nous avons pu réduire notre problème d'apprentissage au simple fait de minimiser une fonction.

Performances

Comme la tâche principale du machine learning consiste à sélectionner un modèle et à l'entraîner sur des données, les deux choses qui peuvent mal tourner sont soit un "mauvais modèle", soit de "mauvaises données".

Validation

Avant de regarder les problèmes énoncés, voyons comment peut-on mesurer si un modèle performe bien ou non.

La première des choses est de séparer notre base de données en deux (usuellement en 80%/20%) :

- Données d'entraînement : données sur lesquelles le modèle apprend
- Données de test : données sur lesquels le modèle final est testé. Ces données ne devront être utilisées qu'à la toute fin pour voir si le modèle performera bien une fois déployer.

Lorsque l'on entraîne un modèle, on cherche également à trouver ces meilleurs hyper-paramètres[6]. Pour ce faire, nous utilisons la validation :

- Ensemble de validation : on redivise notre ensemble d'entraînement en 80%/20% et on test ces hyper-paramètres sur ces 20%
- Cross-validation : on répète plusieurs fois le découpage entraînement/validation de manière aléatoire (souvent préféré pour maximiser les données d'entraînement et réduire la variabilité de la procédure de validation).

Gestion des données

On peut maintenant regarder les problématiques face aux mauvaises données :

1. Manque de données : C'est probablement le problème le plus courant. La plupart des modèles nécessitent des milliers de données (voire des millions pour les réseaux de neurones).
2. Données non représentatives : Les données d'entraînement doivent être les plus représentatives des cas que nous souhaitons généraliser.
3. Mauvaise qualité des données : Trop d'erreurs, de bruits ou d'outliers dans nos données créeront inexorablement un biais dans notre modèle.
4. Features non pertinentes : Le système ne peut apprendre que si les features fournies permettent de caractériser l'output attendu. Pour Zoidberg2.0, la date de création des images ne nous permettrait en rien de classifier nos images.

Underfitting & Overfitting

Côté modèle, les erreurs viennent du dilemme biais / variance.

1. Biais : Il y a un biais lorsque les hypothèses de départ sont erronées. Cela se produit lorsque le modèle est trop simple par rapport aux données fournies. Le modèle aura une grande erreur sur les données d'entraînement. On parle alors de sous-apprentissage (underfitting en anglais).
2. Variance : Une grande variance du modèle correspond à sa grande sensibilité aux légères variations des données d'entraînement. Le modèle va très bien apprendre mais il aura des erreurs importantes sur de nouvelles données. On parle alors de sur-apprentissage (overfitting en anglais).

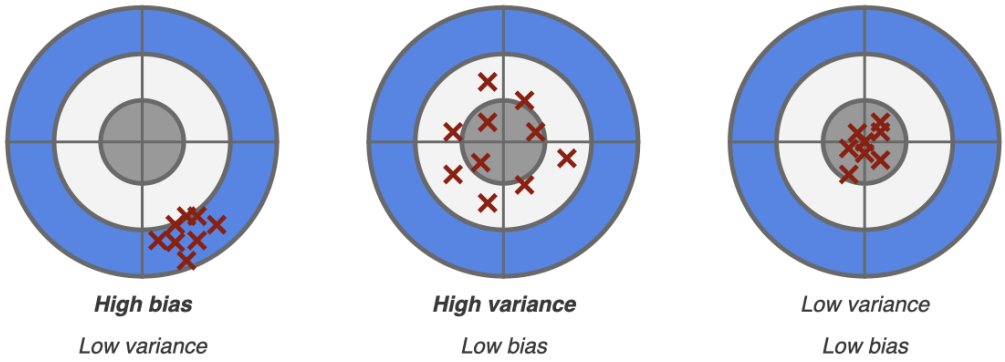


Figure 2. Exemple du dilemme Biais / Variance sur une cible

Dans les deux cas, il existe plusieurs solutions possibles en fonction du modèle utilisé. Voici quelques-unes des solutions possibles :

- Underfitting :
  - Choisir un modèle plus complexe avec plus de paramètres
  - Utiliser des caractéristiques (features) plus adaptées pour le modèle
  - Réduire les contraintes imposées au modèle.
- Overfitting :
  - Choisir un modèle plus simple avec moins de paramètres
  - Restreindre notre modèle avec la régularisation
  - Ajouter plus de données au modèle pour augmenter la variété des exemples
  - Arrêter l'apprentissage du modèle dès qu'il ne progresse plus pour éviter d'apprendre trop spécifiquement les exemples d'entraînement.

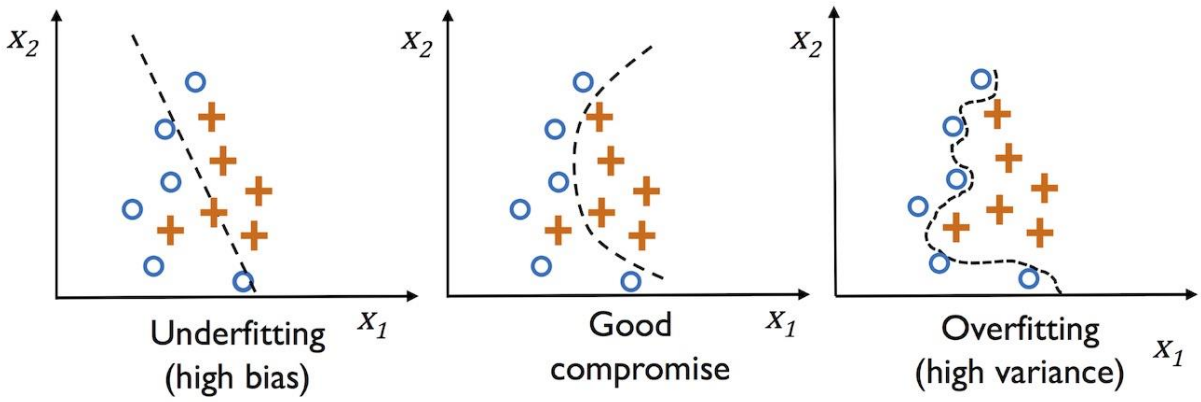


Figure 3. Underfitting et Overfitting dans le cas d'une classification.

Qu'est-ce que le Deep-Learning

Un neurone

Un neurone est une unité de base dans les réseaux de neurones. Il prend en entrée plusieurs features, telles que la couleur ou la taille d'un objet, et renvoie une seule valeur. Cette valeur peut être utilisée pour prédire une catégorie ou une valeur numérique. Le neurone fait passer ces features par 2 étapes :

1. Transformation linéaire : le neurone associe à chaque caractéristique un poids qui reflète son importance dans le calcul de la valeur de sortie. Ces poids θ sont multipliés par les features et additionnés pour produire une valeur intermédiaire.
2. Transformation non-linéaire : la valeur intermédiaire est passée à travers une fonction non-linéaire, appelée fonction d'activation (notée σ), qui produit la valeur de sortie finale. Cette étape est nécessaire car les relations entre les caractéristiques d'entrée ne sont souvent pas linéaires.

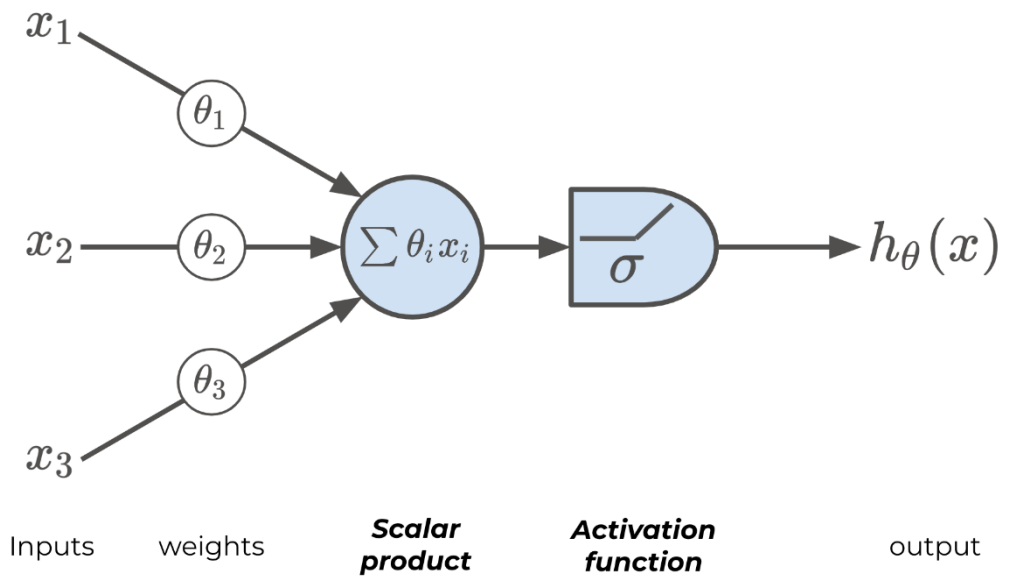


Figure 4. Exemple d'un neurone avec 3 entrées et la fonction ReLu(x) comme fonction d'activation.

Au final, si $x \in R^n$ est le vecteur d'entrée, la sortie d'un neurone est :

$$h_\theta(x) = \sigma(\theta \cdot x + b)$$

Où b est un biais rajouté à chaque neurone[7]. Dans le cas où σ est la fonction sigmoïde, un neurone équivaut à la régression logistique.

Il existe plusieurs fonctions d'activation possibles. Voici les plus utilisées :

- La fonction sigmoïde : $S(x) = \frac{1}{1+e^{-x}}$ qui est utilisée notamment dans la régression logistique. Elle restreint la sortie entre [0, 1].
- La fonction tangente hyperbolique : $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Elle restreint la sortie entre [−1, 1].
- La fonction Rectified Linear Unit : $\mathrm{ReLU}(x) = \max(x, 0)$ et ses dérivées. Elle est très en vogue actuellement du faite que sa sortie soit dans R^+.

Des neurones

Dans un réseau de neurones artificiels, on va simplement utiliser un ensemble de neurones que l'on organise en couches. Les neurones de la couche l utilisent les sorties des neurones de la couche $l-1$ comme entrées. Dans chaque couche l de neurones, nous utilisons généralement la même fonction d'activation σ_l.

Dans notre exemple, nous examinons la couche $l = 1$,qui est la couche la plus basse du réseau de neurones.

Nous notons $\theta_{i,j}^{(1)}$ le poids de l'input x_j pour le neurone i dans la couche $l = 1$. Ainsi, cette couche a comme sortie :

$$a^{(1)} = \sigma_1\left(\Theta^{(1)}x + B\right)$$

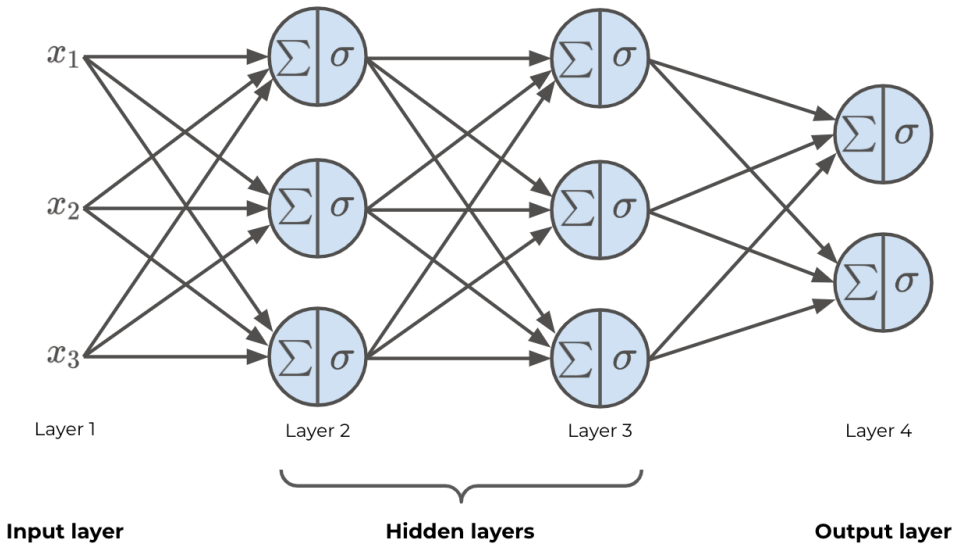


Figure 5. Réseau de neurones entièrement connectés avec 4 couches et une sortie en 2 neurones.

Où $a^{(1)}$ est un vecteur de dimension 3. Ce vecteur sera ensuite utilisé comme entrée pour la couche $l = 2$. La matrice de poids θ est devenue une matrice $\Theta^{(1)}$ de dimensions 3×3^8 dont les éléments sont $\theta_{i,j}^{(1)}$.

Nous pouvons également remarquer que notre réseau se termine avec 2 neurones en sortie. Le nombre de neurones en sortie d'un réseau de neurones dépendra du nombre de classes du problème. Nous aurons un neurone en sortie pour chaque classe. Chaque neurone en sortie renverra un score de probabilité qui représente la probabilité d'appartenance de l'input à cette classe. La somme des scores de tous les neurones de sortie doit être égale à 1, car l'input doit

appartenir à l'une des classes. La classe prédite sera celle ayant le score de probabilité le plus élevé[9].

Enfin, lorsqu'un neurone dans une couche est connecté à tous les neurones de la couche précédente, nous parlons de couche entièrement connectée. Ce type de couche est couramment utilisé dans les réseaux de neurones pour permettre aux neurones de recevoir des informations de toutes les entrées précédentes.

Entraîner un réseau de neurones

L'entraînement des réseaux de neurones a été l'un des défis majeurs en deep learning au XXe siècle. En 1986, D. Rumelhart, G. Hinton et R. Williams ont proposé une solution appelée backpropagation, qui est maintenant largement utilisée.

La backpropagation n'est pas un algorithme d'optimisation en soi, mais plutôt une méthode utilisée pour calculer les gradients du risque par rapport aux poids du réseau de neurones. Ces gradients sont ensuite utilisés par des algorithmes d'optimisation tels que le Gradient Descent pour ajuster les poids du réseau, afin de minimiser le risque.

En reprenant notre exemple de réseau, on a (en omettant le biais) :

$$a^{(1)} = \sigma_1\left(\Theta^{(1)}x\right)$$

$$a^{(2)} = \sigma_2\left(\Theta^{(2)}a^{(1)}\right)$$

$$a^{(3)} = \sigma_3\left(\Theta^{(3)}a^{(2)}\right)$$

avec $h_\Theta(x) = a^{(3)}$ .La chain rule nous permet d'obtenir la dérivée du risque J par rapport à n'importe quel paramètre. Exemple avec $\Theta^{(2)}$, en définissant $z^{(l)} = \Theta^{(l)} \cdot a^{(l-1)}$:

$$\frac{\partial J}{\partial \Theta^{(2)}} = \frac{\partial J}{\partial a^{(3)}} \cdot \frac{\partial a^{(3)}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial \Theta^{(2)}} = \frac{\partial J}{\partial a^{(3)}} \cdot \sigma_3'\left(z^{(3)}\right) \cdot \Theta^{(3)} \cdot \sigma_2'\left(z^{(2)}\right) \cdot a^{(1)}$$

Les termes σ_3' et σ_2' sont simplement les dérivées des fonctions d'activation de la couche 2 et 3, les termes $\Theta^{(3)}$ et $a^{(1)}$ sont calculés lors de la forward pass. Il ne reste que le terme $\partial J / \partial a^{(3)}$ qui est facile à calculer.

Les réseaux de convolution

La dernière partie présente la structure de réseau que nous allons utiliser dans ce projet : les réseaux de convolution. Ce type de réseau est particulièrement performant dans la reconnaissance d'image. Sa performance vient du fait qu'il :

- Permette de réduire le nombre de paramètres et donc le temps d'entraînement[10] en réduisant le nombre de connection d'un neurone et en partageant les mêmes paramètres au sein d'une couche.
- Capture des structures de plus en plus complexes. Leur première couche va capturer des structures très simples dans les images, puis la couche suivante va agréger ces structures pour en produire de plus complexe et ainsi de suite.

Pour cela, il s'appuie sur deux opérations : la convolution & le pooling. Un réseau de convolution est présenté à la figure ci-dessous

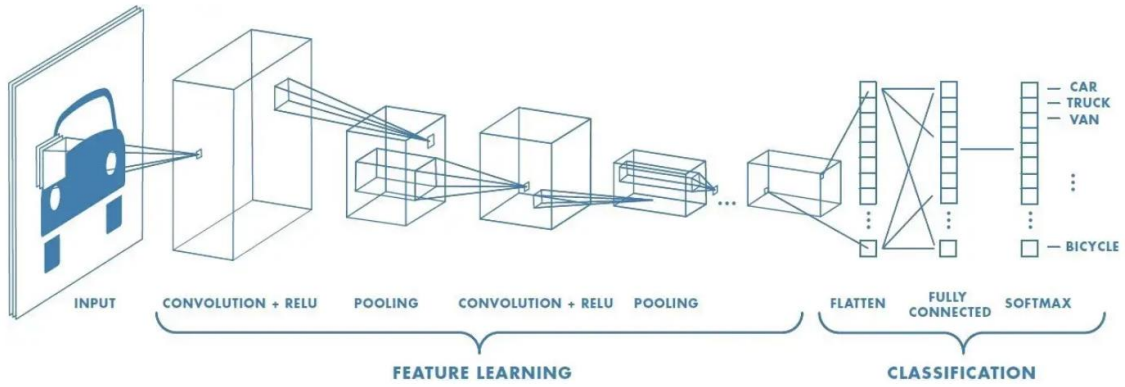


Figure 6. Réseau de convolution typique : la 1 re partie contient successivement des couches de convolution et de pooling. La 2e partie est un réseau entièrement connecté (vu précédemment).

Couche de convolution

Une couche de convolution contient les mêmes éléments qu'une couche entièrement connectée : une transformation linéaire et une transformation non linéaire (dans ce cas-ci, la fonction d'activation est ReLu). Cependant, la différence réside dans la transformation linéaire, le produit scalaire vu dans la section précédente est remplacé par… la convolution !

Dans une couche de convolution, chaque neurone est connecté à une petite partie de la couche précédente appelée champ récepteur. Cela signifie que chaque neurone a un nombre limité de paramètres, déterminé par la taille du champ récepteur. Ces paramètres sont regroupés dans une matrice que l'on appelle kernel. Ce kernel est le même pour tous les neurones de la couche, ce qui permet de réduire le nombre de paramètres à apprendre. En partageant les

mêmes poids, le réseau de neurones peut détecter des motifs similaires à différentes positions de l'entrée sans avoir à apprendre de nouveaux poids pour chaque position. Cette propriété est particulièrement utile pour la détection de motifs spatiaux, tels que les bords, les coins ou les textures, dans une image. Quand on calcule le produit du champ récepteur avec le kernel, on obtient la sortie d'un neurone de convolution. L'ensemble des sorties d'une couche de convolution est appelé feature map.

On peut utiliser plusieurs kernels par couche de convolution, ce qui donnera plusieurs feature maps par couche, ces feature maps étant les inputs de la couche suivante.

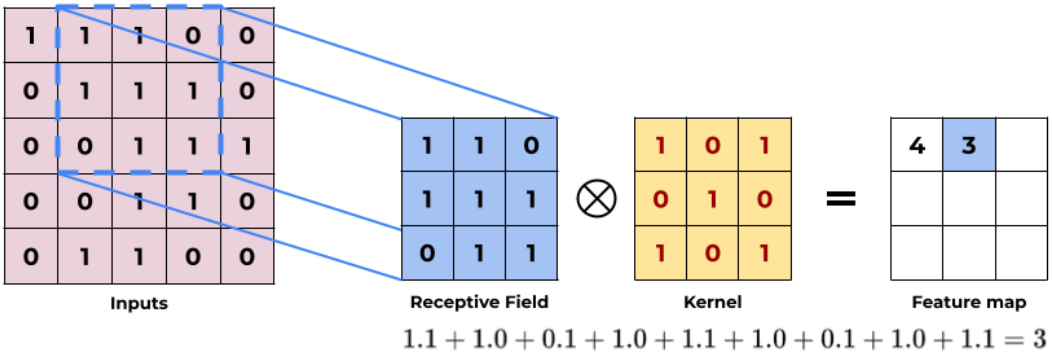


Figure 7. Exemple de convolution avec un champ récepteur de taille 3 × 3.

Couche de pooling

La couche de pooling est plus simple. Son but est d'agréger les données pour réduire le nombre de paramètres. Comme dans la couche de convolution, chaque neurone de la couche de pooling est connecté aux neurones de la couche précédente, présent dans son champ récepteur. Un neurone de pooling va alors agréger les inputs de son champ récepteur via une fonction tel que :

- La fonction max : renvoie l'input le plus élevé
- La fonction avg : renvoie la valeur moyenne des inputs.

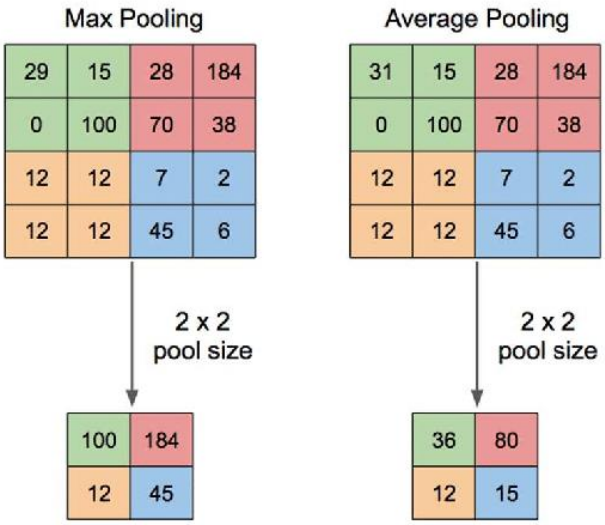


Figure 8. Exemple de pooling où chaque champ récepteur est associé à la couleur de son neurone de pooling

Annexe

TensorFlow[1] : Un cadre d'apprentissage machine open source pour tous

Keras[2] : Une API d'apprentissage profond conçue pour les êtres humains et non pour les machines.

Classification[3] : A noté une équivalence entre classification et régression. En effet, la plupart des modèles s'adaptent très bien aux deux problématiques (SVM, decision tree, etc.).

Entées[4] : En générale, $x_i \in R^n$ i.e. xi est un vecteur de taille n où n correspond au nombre de features (variables caractéristiques de la donnée d'entrée). Dans notre projet, n correspond au nombre de pixels dans les images.

Choisie[5] : La fonction d'hypothèse ne peut pas prendre n'importe quelle forme : elle doit être un concept que l'on peut apprendre et mesurable par la VC dimension.

Risque[6] : En réalité, le risque RA tend vers une valeur optimale et ce que l'on cherche à minimiser est $R_A - R^*$ où $R^{\wedge} *$ est le risque de Bayes. Pour plus de détails, on se référa à l'excellent cours de Francis Bach.

Hyper-paramètres[6] : Les hyper-paramètres du modèle sont des paramètres qui ne sont pas appris par le modèle lui-même, mais qui doivent être choisis avant l'entraînement pour déterminer la structure et le comportement du modèle. Exemples : le learning rate dans le Gradient Descent, le nombre de neurones dans un réseau, etc.

Neurone[7] : Le biais est une valeur ajoutée à un neurone pour compenser les différences dans les données d'entrée.

Dimensions 3×3[8] : Pour chaque couche l, $\Theta^{(l)} \in \mathcal{M}_{n_\ell \times n_{\ell-1}}(R)$ i.e c'est une matrice avec n_{l-1} lignes (nombre de neurones de la couche l) et nl−1 colonnes (nombre de neurones de la couche l − 1).

Élevé[9] : C'est pourquoi nous utilisons généralement en sortie du réseau de neurones, une fonction d'activation spéciale appelée softmax qui normalise les scores de probabilité en une distribution de probabilité sur toutes les classes.

Entraînement[10] : Imaginons une image de 100 × 100 pixels (soit 10 000 features) et une 1er couche entièrement connectée contenant 1 000 neurones, cela représenterait 10 000 000 de paramètres rien que pour cette couche !